

Assignment 3: Geospatial Data Sciences and Economic Spatial Models

Sebastian Dong Uk Paik Sohn, Michael Duarte Gonçalves
Gal · la Gelpí Buxadé, Maria Victoria Suriel Nuñez

2026-02-17

Contents

1	Setup	2
2	Part 1: Climate Change in the USA	3
3	Part 2: Transportation Centrality in Spain	6

1 Setup

We load `gdistance` last because it masks several `tidyverse` functions (`select`, `filter`, `extract`, `union`). To stay safe, we prefix all ambiguous calls with `dplyr::`.

```
library(tidyverse)
library(sf)
library(terra)
library(exactextractr)
library(spData)
library(rnaturalearth)
library(rnaturalearthdata)
library(gdistance)
```

The SPEI dataset lives on the course Dropbox. We only download it once.

```
dir.create("data", showWarnings = FALSE)

spei_path <- "data/spei01.nc"

if (!file.exists(spei_path)) {
  message("Downloading SPEI data (~340 MB)...")
  download.file(
    url = str_c(
      "https://www.dropbox.com/scl/fi/",
      "kmo2gj0iqvu52rat9ydmf/",
      "spei01.nc?rlkey=1ahg7k4cs1uf1v52s5x6d0sxj",
      "&dl=1"
    ),
    destfile = spei_path,
    mode     = "wb"
  )
}
```

2 Part 1: Climate Change in the USA

2.1 Preparing the regions

We start from `us_states` (`spData`), drop Alaska and Hawaii, and dissolve state boundaries into the four Census regions.

```
# Keep continental states with their region label
sf_states <- us_states %>%
  dplyr::filter(!NAME %in% c("Alaska", "Hawaii")) %>%
  dplyr::select(NAME, REGION)

# Merge states into region-level polygons
sf_regions <- sf_states %>%
  group_by(REGION) %>%
  summarise(geometry = st_union(geometry)) %>%
  ungroup()
```

2.2 Loading the SPEI raster

The NetCDF has 1,380 monthly layers (Jan 1901 - Dec 2015). We keep all months from 1966 to 2015 and compute the yearly average, giving us a clean 50-year annual panel.

```
r_spei <- rast(spei_path)
spei_dates <- time(r_spei)

# Keep all monthly layers from 1966 to 2015
layer_index <- which(
  year(spei_dates) >= 1966 &
  year(spei_dates) <= 2015
)

r_spei_monthly <- r_spei[[layer_index]]

# Compute yearly averages (mean of 12 months per year)
spei_years_all <- year(time(r_spei_monthly))
r_spei_50 <- tapp(
  r_spei_monthly, spei_years_all, fun = mean
)

spei_years <- names(r_spei_50) %>%
  str_extract("\\d{4}") %>%
  as.integer()

names(r_spei_50) <- str_c("spei_", spei_years)
```

2.3 Zonal statistics

We compute the area-weighted mean SPEI within each region for every year. This gives us a panel dataset, essentially one number per region per year, which we then reshape into long (tidy) format for plotting.

```
# Area-weighted mean SPEI per region per year
df_zonal <- exact_extract(
  x = r_spei_50,
  y = sf_regions,
```

```

    fun = "mean"
  )

## |
# Pivot into a tidy panel: one row per region-year
df_panel <- df_zonal %>%
  mutate(REGION = sf_regions$REGION) %>%
  pivot_longer(
    cols = starts_with("mean.spei_"),
    names_to = "year",
    values_to = "spei"
  ) %>%
  mutate(
    year = str_extract(year, "\\d{4}") %>%
      as.integer()
  )

```

2.4 Figure 1: SPEI evolution by region

We plot the SPEI for each Census region over the 50-year window. Positive values indicate wetter-than-normal conditions, while negative values signal drought. We overlay a LOESS smoother (in blue) to capture the overall trend across all four regions.

```

ggplot(
  df_panel,
  aes(x = year, y = spei, color = REGION)
) +
  geom_line(alpha = 0.7, linewidth = 0.6) +
  geom_smooth(
    aes(group = 1),
    method = "loess",
    color = "blue",
    linewidth = 1.2,
    se = TRUE
  ) +
  labs(x = "year", y = "spei", color = "REGION") +
  theme_bw()

```

```
## `geom_smooth()` using formula = 'y ~ x'
```

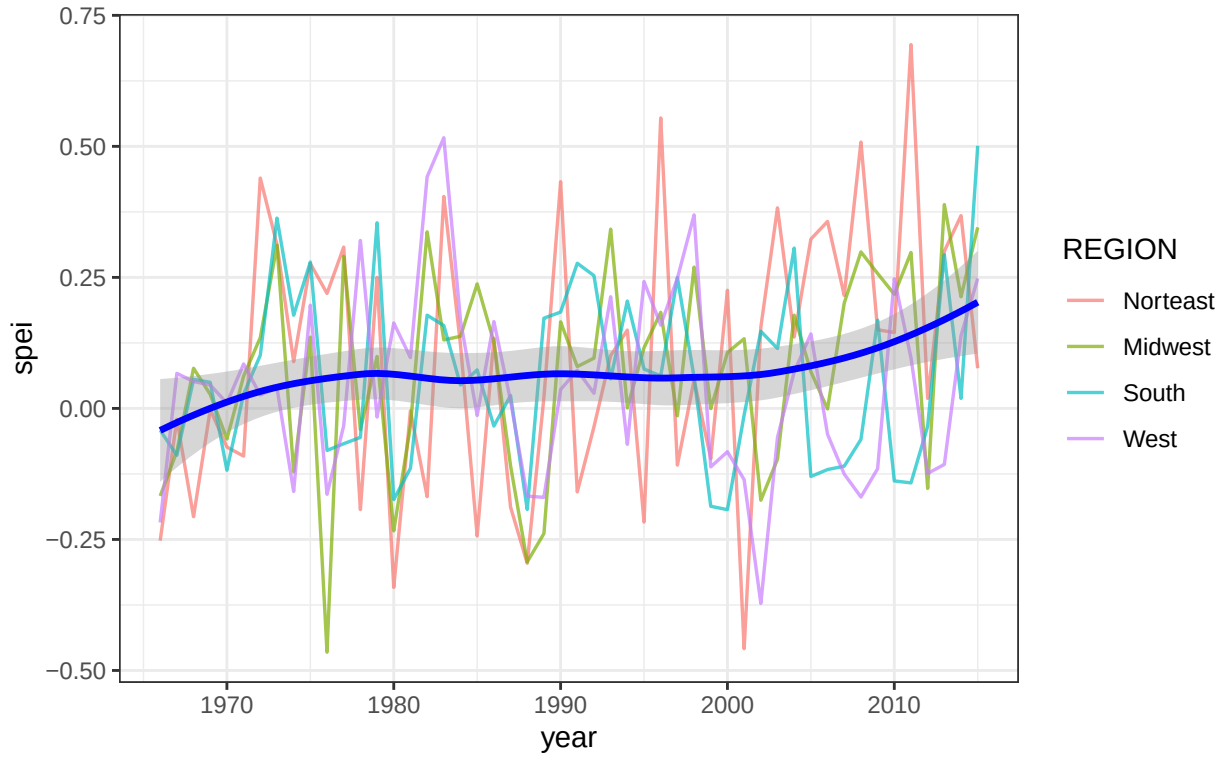


Figure 1: SPEI by region.

Several patterns stand out. First, we observe a gentle downward slope in the LOESS trend, suggesting a modest drying tendency over the half-century. Second, individual drought episodes, like the mid-1970s, the late 1980s, appear simultaneously in multiple regions, pointing to large-scale climate drivers rather than purely local phenomena. Overall, the figure illustrates that while year-to-year variability dominates, there is a slow structural shift toward drier conditions that matters for long-run agricultural planning and water resource management.

3 Part 2: Transportation Centrality in Spain

3.1 Geographic setup

We grab Spain's boundary from `spData::world`, then download Natural Earth populated places and roads at 10m resolution. Column names come in lowercase from the `.gpkg` format, so we normalize just in case.

```
sf_spain <- world %>%
  dplyr::filter(name_long == "Spain") %>%
  st_transform("EPSG:4326")

# Populated places
sf_places_raw <- ne_download(
  scale      = 10,
  type       = "populated_places",
  category   = "cultural",
  destdir    = "data",
  returnclass = "sf"
)

# Ensure lowercase columns (gpkg vs shp inconsistency)
names(sf_places_raw) <- tolower(names(sf_places_raw))

# Top 10 Spanish cities by population
sf_places <- sf_places_raw %>%
  dplyr::filter(iso_a2 == "ES") %>%
  arrange(desc(pop_max)) %>%
  slice_head(n = 10) %>%
  dplyr::select(name, pop_max)

# Road network, cropped to Spain
sf_roads_raw <- ne_download(
  scale      = 10,
  type       = "roads",
  category   = "cultural",
  destdir    = "data",
  returnclass = "sf"
)

sf_roads_spain <- sf_roads_raw %>%
  st_crop(st_bbox(sf_spain)) %>%
  st_intersection(sf_spain)
```

3.2 Friction surface

We rasterize the road network at 0.1 degree resolution. Road cells receive a value of 1, while all other cells get 1/100, meaning off-road travel is 100 times more costly. This is a deliberately simple model: we are not claiming it captures actual travel times, but it gives us a reasonable approximation of how the road network shapes connectivity.

```
# Template raster covering Spain
r_template <- rast() %>% crop(sf_spain)
res(r_template) <- 0.1

# Burn roads into the grid
r_roads <- rasterize(
```

```

    vect(sf_roads_spain), r_template
  )

  # Off-road pixels become 1/100 (costly but not impossible)
  vv <- values(r_roads)
  vv[is.nan(vv) | is.na(vv)] <- 1 / 100
  values(r_roads) <- vv
  rm(vv)

```

3.3 Figure 2: Friction surface

```

plot(r_roads, main = "Friction Surface - Spain")
plot(
  st_geometry(sf_spain),
  add = TRUE, border = "black", lwd = 2
)
plot(
  st_geometry(sf_places),
  add = TRUE, pch = 20, cex = 1.5
)

```

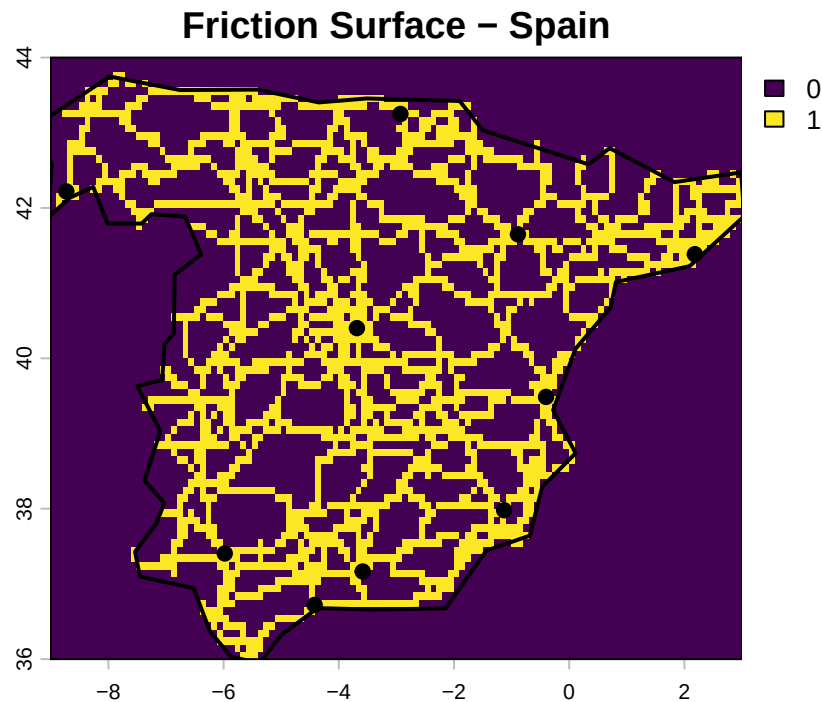


Figure 2: Friction surface.

3.4 Transition matrix and bilateral distances

We convert our friction raster into a transition matrix using `gdistance`. We apply a geographic correction to account for the fact that grid cells at different latitudes have different physical sizes. Then we compute all pairwise distances in a single call with `costDistance()`. We do this because it is much faster than looping `shortestPath()` over every pair individually.

```

# gdistance needs the old raster format
tr_matrix <- transition(
  x = raster::raster(r_roads),
  transitionFunction = mean,
  directions = 8
) %>%
  geoCorrection(type = "c")

# All pairwise distances in one call
dist_matrix <- costDistance(
  tr_matrix,
  as(sf_places, "Spatial")
) %>%
  as.matrix()

# Label and convert to km
city_names <- sf_places$name
rownames(dist_matrix) <- city_names
colnames(dist_matrix) <- city_names

dist_matrix_km <- dist_matrix / 1000
round(dist_matrix_km, 0)

```

```

##           Madrid Barcelona Seville Bilbao Valencia Zaragoza Málaga Murcia
## Madrid           0         582      453      357         340        301      447      379
## Barcelona       582           0      940      538         346        300      874      540
## Seville          453          940        0      787         599        754      182      492
## Bilbao           357          538      787         0         541        290      805      731
## Valencia         340          346      599      541           0        290      533      194
## Zaragoza         301          300      754      290         290         0        735      483
## Málaga           447          874      182      805         533        735         0      363
## Murcia           379          540      492      731         194        483        363         0
## Granada          404          790      238      762         449        658       122      279
## Vigo            637          1146      919      612         976        846      1057     1016
##           Granada Vigo
## Madrid          404    637
## Barcelona       790  1146
## Seville         238    919
## Bilbao          762    612
## Valencia        449    976
## Zaragoza        658    846
## Málaga          122  1057
## Murcia          279  1016
## Granada          0  1025
## Vigo           1025     0

```

3.5 Figure 3: Madrid–Vigo shortest path

While `costDistance()` gives us the numbers, we also want to see the actual route on a map. We use `shortestPath()` for the Madrid-Vigo pair specifically, which returns a line geometry we can overlay on the road network.

```

sf_madrid <- sf_places %>%
  dplyr::filter(name == "Madrid")
sf_vigo <- sf_places %>%

```



```

dplyr::filter(name == "Vigo")

# shortestPath gives us the line geometry for plotting
path_mv <- shortestPath(
  x      = tr_matrix,
  origin = st_coordinates(sf_madrid),
  goal   = st_coordinates(sf_vigo),
  output = "SpatialLines"
)

# Convert to sf and assign CRS (shortestPath drops it)
sf_path_mv <- path_mv %>%
  st_as_sf() %>%
  st_set_crs(4326)

ggplot() +
  geom_sf(
    data = sf_spain,
    fill = "grey90",
    color = "black"
  ) +
  geom_sf(
    data      = sf_roads_spain,
    color     = "grey40",
    linewidth = 0.3
  ) +
  geom_sf(
    data      = sf_path_mv,
    color     = "red",
    linewidth = 1
  ) +
  geom_sf(
    data = sf_places,
    size = 2,
    color = "black"
  ) +
  geom_sf(
    data = sf_madrid,
    size = 4,
    color = "red"
  ) +
  geom_sf(
    data = sf_vigo,
    size = 4,
    color = "blue"
  ) +
  geom_sf_label(
    data      = sf_madrid,
    aes(label = name),
    nudge_y = 0.3
  ) +
  geom_sf_label(
    data      = sf_vigo,
    aes(label = name),

```

```

nudge_y = 0.3
) +
labs(
  title = "Road Network & Shortest Path: Madrid - Vigo"
) +
theme_minimal()

```

```

## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data
## Warning in st_point_on_surface.sfc(sf::st_zm(x)): st_point_on_surface may not
## give correct results for longitude/latitude data

```

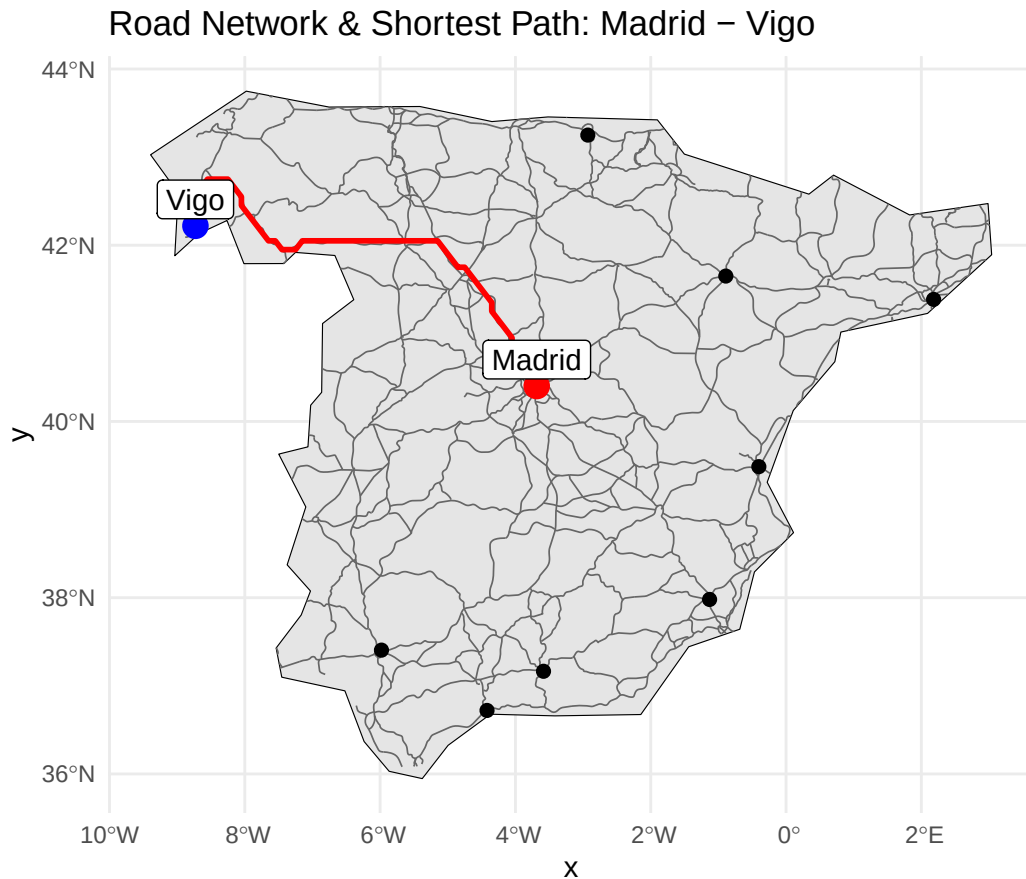


Figure 3: Madrid–Vigo path.

```

sf_path_mv %>%
  st_length() %>%
  `/(1000)` %>%
  round(0) %>%
  str_c(
    "Madrid-Vigo network distance: ", ., " km"
  ) %>%
  cat("\n")

```

```

## Madrid-Vigo network distance: 622 km

```

3.6 Comparing isolation

We extract each city's row from the distance matrix. We then stack the two sets of distances into a tidy tibble so we can compare them side by side.

```
idx_madrid <- which(city_names == "Madrid")
idx_vigo   <- which(city_names == "Vigo")

df_distances <- bind_rows(
  tibble(
    origin = "Madrid",
    dest   = city_names[-idx_madrid],
    distance = dist_matrix_km[idx_madrid, -idx_madrid]
  ),
  tibble(
    origin = "Vigo",
    dest   = city_names[-idx_vigo],
    distance = dist_matrix_km[idx_vigo, -idx_vigo]
  )
)
```

3.7 Figure 4: Distance distributions

We now compare how far each city is from every other top-10 city. The density plot tells us, at a glance, whether a city sits at the centre of the network or out on the periphery. If a city's curve is concentrated on the left, most other cities are nearby – that city is well connected. If the curve shifts to the right, the city is structurally more isolated.

```
ggplot(
  df_distances,
  aes(x = distance, fill = origin)
) +
  geom_density(alpha = 0.5) +
  xlim(0, 1500) +
  scale_fill_manual(
    values = c(
      "Madrid" = "orangered",
      "Vigo"   = "forestgreen"
    )
  ) +
  labs(
    title = "Transportation Centrality: Madrid vs. Vigo",
    x     = "Distance (km)",
    y     = "Density",
    fill  = "Origin"
  ) +
  theme_minimal()
```

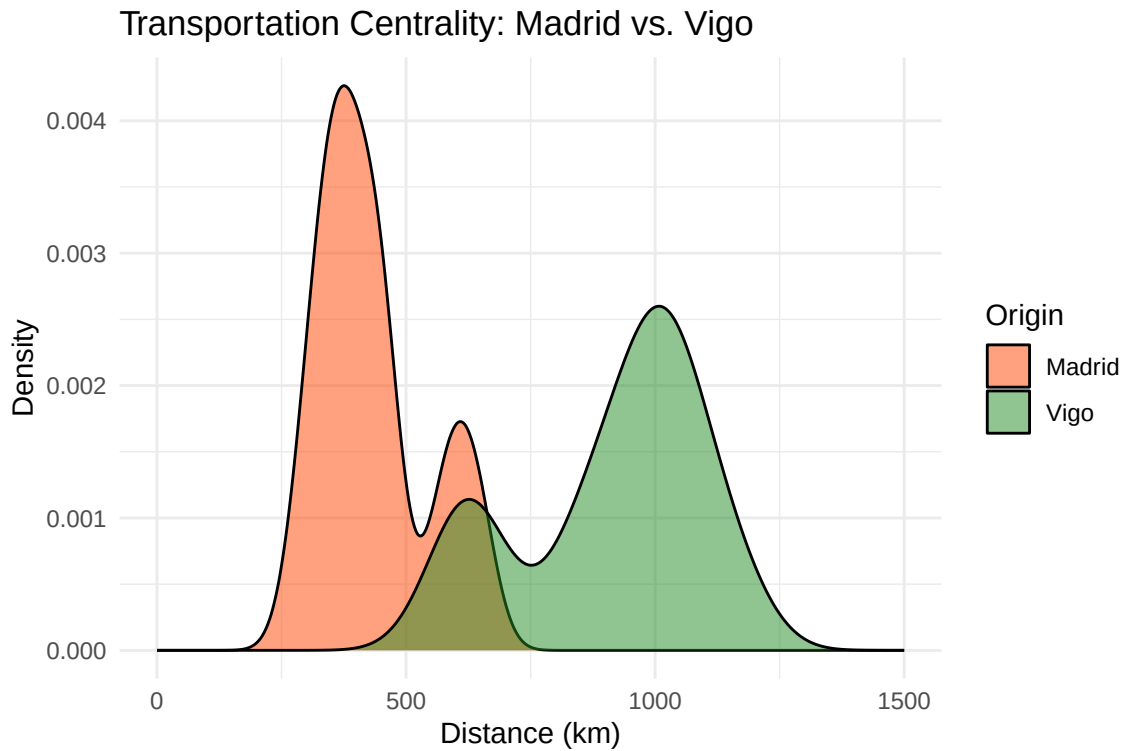


Figure 4: Madrid vs Vigo.

The result is striking. We see that Madrid's distribution peaks at shorter distances and is more tightly concentrated, reflecting its position at the center of Spain's radial highway system. Most major cities are reachable within a relatively narrow band. Vigo's distribution, by contrast, is shifted substantially to the right, every other top-10 city is farther away, and the spread is wider.

We can interpret the gap between the two curves as a measure of differential market access. Firms in Madrid can reach a larger consumer base at lower transport cost than firms in Vigo. This structural advantage helps explain Madrid's dominance as a logistics and services hub, while Vigo, despite being an important Atlantic port, remains more regionally oriented.